

13. Education Page — ASL Stickman Visualizations

Purpose: This section documents the Education tab of the application (the “stickman” pages) at the same technical depth as the Translation & Chat features described earlier in this chapter: its two-panel layout, the shared Canvas2D rendering engine that drives both panels, the keyframe/inverse-kinematics animation system used for full-body word signs, the procedurally generated fingerspelling hand-pose library, and the performance and correctness engineering decisions specific to this feature.

Unlike the Chatting tab, the Education tab performs no server round-trip: it is a fully client-side, dependency-light feature built entirely on the browser's native Canvas 2D API and plain JavaScript. Its purpose is instructional — helping a user learn to read ASL handshapes and word signs — rather than translating a live camera feed.

13.1 Overview and Layout

Purpose: To describe how the Education tab is composed and how it fits into the existing App Home Page navigation.

The Education tab is one of the two primary tabs on the App Home Page, alongside Chatting. Selecting it swaps the Chatting panel for a “Learn to Sign” section that is itself split into two side-by-side columns inside a responsive grid layout:

- **Left column — Fingerspelling • A-Z:** renders the ASLStickman component. A user types any word into a text field and the stickman spells it out letter by letter, with the current letter, its distinguishing cue, and a live alphabet reference grid shown alongside the animation.
- **Right column — Word Signs • Full Body:** renders the ASLWordStickman component. A user picks one of fifteen common everyday words (COME, DRINK, EAT, GO, HELLO, HELP, LEARN, MORE, NAME, NO, PLEASE, SORRY, STOP, WATER, YES) from a button grid and watches a full-body stick figure perform the sign, at a Slow (1.0×) or Normal (0.55× duration) playback speed.

Both columns share an introductory line of copy above the grid explaining the two practice modes, and each column carries its own short instructional subtitle so the two panels can be understood independently.

13.2 Motivation: Replacing a Heavy Third-Party Renderer

Purpose: To record why the education page was rebuilt as a bespoke Canvas2D renderer rather than continuing to rely on an imported animation library.

An earlier iteration of the word-sign panel rendered full-body signs through a third-party <pose-viewer> web component — a Stencil/WebGL-based custom element loading external .pose files, roughly 20 MB of dependency weight. In practice this made the Education tab noticeably laggy and occasionally desynchronised from the correct ASL handshape, since the component's bundled pose data did not

always match the sign being requested. Both stickman panels were therefore rewritten from scratch as pure Canvas 2D drawing routines with no external animation runtime: the entire rendering stack — palette, easing curves, interpolation helpers, inverse kinematics, and the hand-drawing routine — is implemented in a single shared utility module (`aslRenderer.js`) with zero third-party dependencies. This keeps the panel smooth even on modest laptop hardware, and keeps the rendered handshape and the underlying landmark data as one single source of truth.

13.3 Shared Rendering Engine (`aslRenderer.js`)

Purpose: To describe the low-level drawing primitives reused by both the fingerspelling and word-sign panels.

Both stickman components are built on one shared, framework-free utility module that exposes:

- **Colour palette (COLORS):** assigns semantic meaning to each visual element — violet for the static body/torso/left arm, pink for the active signing arm, amber for the thumb (so palm orientation reads instantly), pale yellow for knuckle joints, and white for fingertips. The same convention is used consistently across both panels so a learner does not have to re-learn a colour code between fingerspelling and word-sign practice.
- **Hand landmark layout (HAND_CONNECTIONS):** a MediaPipe-compatible 21-point layout — landmark 0 is the wrist; landmarks 1–4, 5–8, 9–12, 13–16 and 17–20 are the thumb, index, middle, ring and pinky chains respectively — matching the same landmark convention used by the live-translation letter models, so hand-pose data is conceptually interchangeable between the recognition pipeline and the teaching visualisation.
- **Easing table (EASE):** linear, easeIn, easeOut, easeInOut and easeOutBack cubic/back-easing curves used to time keyframe transitions in the word-sign animation.
- **Interpolation helpers:** lerp, lerpXY, and lerpPose — the last of which linearly blends every one of the 21 landmarks between two full hand poses simultaneously, producing the smooth “morphing” effect seen when the fingerspelling stickman transitions between two letters.
- **Two-joint inverse-kinematics solver (solveArm):** used only by the word-sign panel; see Section 13.6.
- **Batched hand-drawing routine (drawHand):** anchors landmark 0 at a given canvas point and scales the remaining 20 landmarks around it, so the same 21-point pose data can be rendered at a fixed wrist (fingerspelling) or at a wrist that itself moves under inverse kinematics (word signs). Drawing is batched per visual layer — one filled path for the palm outline, one stroke pass for all non-thumb finger bones, one stroke pass for the thumb, one fill pass for every knuckle, one fill pass for every fingertip — rather than issuing a separate canvas call per bone or joint, keeping each animation frame cheap to render.

13.4 Fingerspelling Panel (`ASLStickman`)

Purpose: To describe how arbitrary typed text is converted into a sequence of animated ASL letter handshapes.

The left-hand panel lets a user type any word (up to 40 characters) and press Play. The component:

1. Filters the typed text down to the characters it can render — the 26 letters plus a space glyph representing a neutral/rest handshape — via the shared ASL_POSES lookup table.
2. Draws a static stick figure (head, torso, legs, a relaxed left arm) with the right arm permanently raised toward a fixed wrist position, so only the hand itself needs to animate.
3. Steps through the filtered letter sequence one at a time. Each letter has two animation phases: a 260 ms morph, during which lerpPose blends all 21 landmarks from the previous letter's handshape into the new one using an ease-in-out curve, followed by a 620 ms hold during which the completed handshape is held still so it can be read.
4. Overlays the current letter in large text above the head, and renders a caption drawn from ASL_HINTS — a short, single distinguishing cue per letter — plus a row of progress dots indicating how far through the typed word the animation has advanced.
5. Special-cases the two ASL letters that are not static poses — J and Z — via a MOTION_LETTERS list. For these two letters the underlying handshape is held fixed (J reuses the I handshape; Z uses a pointing index finger) while a small time-based wrist offset traces the letter's characteristic motion in the air: J traces a downward hook-and-curl, and Z traces the three strokes of the letter itself, evaluated as a piecewise function of the hold phase's progress.

Below the animation canvas the panel also renders a 26-button A-Z reference grid; tapping any letter (when not currently playing a word) freezes the stickman on that single handshape and its hint text, letting a learner browse the whole alphabet independently of typing a word.

13.5 Full-Body Word-Sign Panel (ASLWordStickman)

Purpose: To describe how a single ASL word gloss is expressed as a keyframed, full-body animation.

The right-hand panel is driven by a small per-word animation library (`aslWordPoses.js`) rather than a single static handshape. Each of the fifteen supported words maps to a `WORD_ANIMS` entry consisting of a short human-readable description and an ordered array of keyframes. Every keyframe specifies, for either or both hands, a target wrist position on the 320×500 canvas, a target handshape (drawn from the same ASL_POSES letter/handshape table used by the fingerspelling panel, since ASL word signs are frequently built from the same underlying handshapes as the manual alphabet), optional vertical offsets for the head and shoulders (used for signs that involve a lean or a shrug), a move duration, a hold duration, and a named easing curve.

At playback time the component:

- Walks the keyframe array with `requestAnimationFrame`, tracking, for each keyframe, how much of its declared move duration and hold duration (both scaled by the selected Slow/Normal speed multiplier) has elapsed.
- During the move phase, linearly interpolates each wrist from the previous keyframe's resting position toward the current keyframe's target using that keyframe's own easing curve, and separately snaps the rendered handshape over to the new target at the 50%-progress mark of the

move — rather than at its start or end — so a handshape change reads as occurring mid-transition rather than as an abrupt pop at either boundary.

- During the hold phase, freezes on the keyframe's exact target position and handshape so the pose can be read.
- Returns both wrists to a shared neutral rest position at the sides of the body at the end of every completed word, and shows a translucent “▶ Press Play to sign” / “↺ Press Play again” caption on the idle canvas.
- Renders a row of progress dots along the bottom edge of the canvas, one per keyframe, highlighting the currently active keyframe in the accent colour while it is in its hold phase.

13.6 Two-Joint Inverse Kinematics and Elbow Continuity

Purpose: To explain how a single wrist target is converted into a visually natural elbow bend for both arms.

Because each keyframe only specifies a wrist target — not an elbow position — the stickman's upper-arm and forearm segments (each a fixed-length bone from a shoulder anchored at a fixed point) are resolved every frame by a two-bone (shoulder → elbow → wrist) inverse-kinematics solver, `solveArm`. Given a shoulder point, a wrist target, the two bone lengths, and a “hint” point, the solver:

6. Clamps the shoulder-to-wrist distance so it never exceeds the sum of the two bone lengths or drops below their difference, which keeps the triangle solvable even for a keyframe whose author placed the wrist slightly out of the arm's true reach.
7. Uses the law of cosines to find the elbow angle relative to the shoulder–wrist axis, which — because the law of cosines only determines an angle's magnitude — always yields two mathematically valid elbow positions, one on either side of the shoulder–wrist line.
8. Picks whichever of the two candidate elbow positions is closer to the supplied hint point.

A naive implementation would supply a single fixed hint per arm (the module does define static defaults that keep the elbows bending forward and outward in a natural resting direction), but doing so for every frame of a fast animation can make the elbow visibly “teleport” between the two IK solutions whenever the wrist crosses the line along which both solutions are equally close to the fixed hint. The word-sign player instead feeds each frame's own resolved elbow position forward as the next frame's hint — only the very first frame of a playback falls back to the static default — so the elbow always continues bending in whichever direction it was already bending, eliminating discontinuous flips mid-sign.

13.7 Hand Pose Library (`aslHandPoses.js`)

Purpose: To document how the 21-landmark handshape for every fingerspelling letter is constructed and verified.

Rather than hand-authoring 21 raw coordinate pairs per letter, each handshape in `ASL_POSES` is built procedurally from a small set of anatomical building blocks:

- Four fixed palm-anchor points (the wrist, the thumb's base joint, and the index/middle/ring/pinky knuckle positions).
- Per-finger segment lengths for the three bones of each of the four non-thumb fingers.
- A `seg4()` helper that walks outward from a knuckle along a base direction (in degrees) and applies a cumulative joint-flex offset at each of the three joints, returning the finger's [MCP, PIP, DIP, TIP] points.
- A small named library of reusable curl patterns — straight, a gentle curve, a half-curl that meets the thumb, a tight fist, a hook bent mainly at the middle joint, and a finger draped over the thumb — so that visually related letters reuse the same curl constants rather than each defining bespoke joint angles.
- A dedicated `thumb()` helper, since the thumb has only three bone segments and moves largely independently of the curl patterns above.

The resulting `pose()` call concatenates the wrist with all five fingers into the same 21-point MediaPipe ordering used everywhere else in the system. Each handshape additionally carries a one-line entry in `ASL_HINTS` — a single, specific visual cue (for example, which finger is extended, or how the thumb is positioned relative to the fist) — that is surfaced as an on-canvas caption during both the alphabet reference grid and the fingerspelling animation, so a learner is told what to look for rather than only being shown a picture.

Handshape and word-sign correctness were treated as a first-class engineering concern rather than a one-off design pass. Early iterations of the word-sign library were built primarily from general recollection of ASL and were found, on review, to contain a number of signs that did not match authoritative usage — an incorrect handshape, an incorrect number of hands, or a location or motion that did not match the real sign. The animation data was subsequently revised through a structured verification pass: each word's motion, handshape(s) and hand count were checked against established ASL reference material, and any sign flagged as incorrect was corrected in `aslWordPoses.js` and re-verified by rendering and visually inspecting the animation frame-by-frame, since some defects — such as two hands rendering close enough together to visually merge into an indistinguishable blob, or a keyframe wrist coordinate placed just outside the head's drawn radius so a sign meant to touch the forehead instead appeared to float beside it — are geometry bugs that only surface once the sign is actually rendered, not when its written description is reviewed in isolation.

13.8 Performance Engineering

Purpose: To record the specific rendering decisions that keep the Education tab responsive.

Two deliberate choices distinguish this feature's performance profile from a typical continuously animated canvas widget:

- **No perpetual animation loop.** Both stickman components request an animation frame only while a sign is actively being played. When idle — including immediately after a sign finishes, and whenever the selected word or preview letter changes — each component instead paints a single static frame and stops, rather than running `requestAnimationFrame` indefinitely in the background. This means

the Education tab consumes essentially no CPU while a user is reading a completed sign or choosing what to play next, and the animation loop is always explicitly cancelled both on Stop and on component unmount to avoid leaking a frame callback.

- **A single cached 2D rendering context.** The canvas's `getContext("2d")` result is captured once, in a ref, on mount rather than being re-queried on every frame.

Combined with the removal of the third-party pose-viewer dependency described in Section 13.2, in favour of a small set of dependency-free Canvas2D components and shared data/utility modules, this rewrite was the direct fix for the education page's original lag: the panel now performs comparably to any other lightweight canvas UI element regardless of how long a user leaves it open.

13.9 UX and Accessibility Considerations

Purpose: To summarise the interaction-design choices that make the two panels usable as a self-serve learning tool.

- **Consistent, theme-aware colour language.** Both panels draw from the same CSS custom-property palette used throughout the rest of the application, so the Education tab automatically follows the app-wide dark/light theme toggle rather than maintaining its own hard-coded styling.
- **Guarded interaction states.** Word and letter selection buttons, and the alphabet reference grid, are disabled while an animation is actively playing, preventing a user from queuing a second sign mid-playback and leaving the component's internal animation state inconsistent with what is on screen.
- **Always-visible progress and captioning.** Both panels continuously surface where the viewer is within the current sign — a highlighted progress dot for the active letter or keyframe — alongside a short plain-language caption (the letter's distinguishing cue, or the word's overall description), so the visualisation functions as a teaching aid rather than only an animation.
- **Responsive canvas sizing.** Both canvases are declared at a fixed drawing resolution (340×430 for fingerspelling, 320×500 for word signs) but are constrained to a maximum width of 100% in the surrounding layout, so the panel scales down cleanly on narrower viewports without needing separate mobile-specific rendering logic.

13.10 Summary

The Education tab's two stickman panels — fingerspelling (ASLStickman) and full-body word signing (ASLWordStickman) — are built entirely on a shared, dependency-free Canvas2D rendering module (`aslRenderer.js`) and two data libraries of procedurally constructed ASL handshapes (`aslHandPoses.js`) and keyframed word-sign animations (`aslWordPoses.js`). A two-joint inverse-kinematics solver with frame-to-frame elbow continuity gives the word-sign panel natural-looking arm motion from nothing more than a sequence of wrist targets, while shared interpolation and easing helpers drive smooth handshape morphing in both panels. The rewrite that replaced an earlier, heavier third-party pose-viewer dependency — combined with rendering only while actively animating, rather than in a perpetual loop — is what makes the Education tab responsive, and a structured, render-and-inspect verification pass over the word-sign keyframe data is what keeps the signs it teaches accurate.